

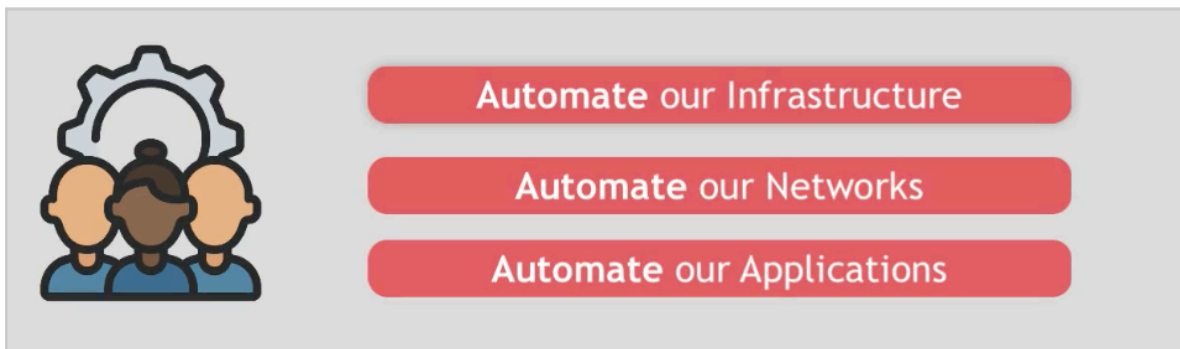
Module 15 - chapter 24

Ansible Roles - Make your Ansible content more reusable and modular

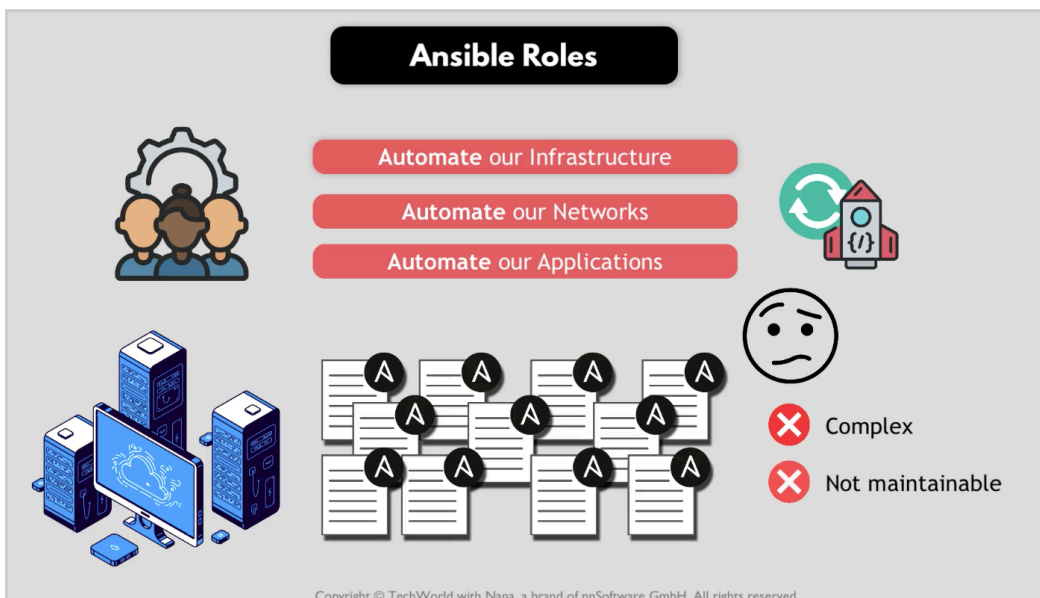
The final YAML file used in the project can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/ansible-projects/-/blob/main/deploy-docker-with-roles.yaml>

Ansible Roles

Let's say our organisation is adopting ansible and it is using it to automate different tasks:



And the infra is complex so very quickly the ansible files get complex as well with lots of playbooks and lots of tasks.



It becomes difficult to have an overview of what all the ansible playbooks actually do.

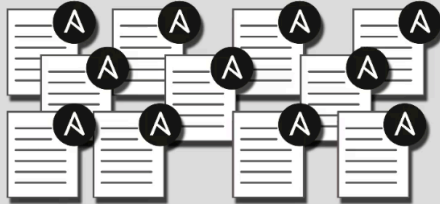
This is when ROLES come into the picture.

Roles allow us to structure our complex ansible project with lots of playbacks

and tasks.

Ansible Roles

- ▶ Group your content in roles
- ▶ To easily reuse and share them with others
- ▶ Break up large playbooks into smaller manageable files






✗

Complex

✗

Not maintainable

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Role is a package for our tasks in a play

```
- name: Create new linux user
hosts: all
become: yes
tasks:
  - name: Create new linux user
    ansible.builtin.user:
      name: nana
      groups: adm,docker

- name: Start docker containers
hosts: all
become: yes
become_user: nana
vars_files:
  - project-vars
tasks:
  - name: Copy docker compose
    ansible.builtin.copy:
      src: /Users/nana/bootcamp-java-mysql-project/docker-compose-full.yaml
      dest: /home/nana/docker-compose.yaml
  - name: Docker login
    community.docker.docker_login:
      registry_url: https://index.docker.io/v1/
      username: nanajanashia
      password: "{{docker_password}}"
  - name: Start container from compose
    community.docker.docker_compose:
      project_src: /home/nana
```

main.yaml 397 Bytes

```
1 - name: Copy docker compose
2   ansible.builtin.copy:
3     src: docker-compose.yaml
4     dest: /home/nana/docker-compose.yaml
5
6 - name: Docker login
7   community.docker.docker_login:
8     registry_url: "{{ docker_registry }}"
9     username: "{{ docker_username }}"
10    password: "{{ docker_password }}"
11
12 - name: Start container from compose
13   community.docker.docker_compose:
14     project_src: /home/nana
```

We can take the tasks from the play and package them in a small bundle with is a ROLE 🙌

So if multiple plays use the same tasks we can reuse the role in multiple plays. We have make the task reusable via role packages.

Ansible Roles

```

- name: Create new linux user
  hosts: all
  become: yes
  vars:
    user_groups: adm,docker
  roles:
    - create_user

- name: Start docker containers
  hosts: all
  become: yes
  become_user: nana
  vars_files:
    - project-vars
  roles:
    - start_containers

```

✓

Re-use roles in different Plays

✓

Much cleaner Plays

Usage of roles in your Play

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Roles are not only for tasks!

We can also package things that the tasks need inside the role.

For example static files where the task is about moving files from one server to another or from the control node to managed nodes

Ansible Roles

✓

Tasks that the role executes

✓

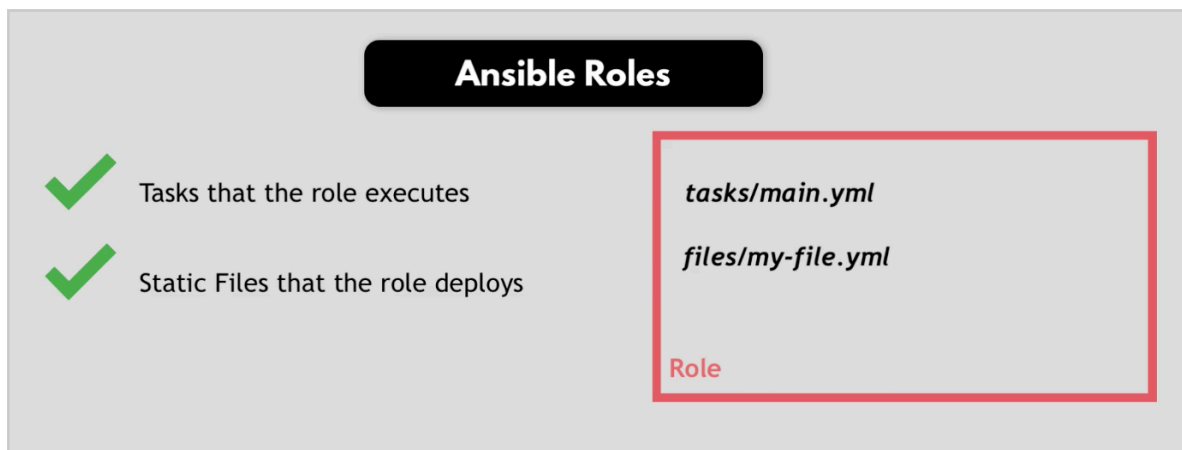
Static Files that the role deploys

files/my-file.yml

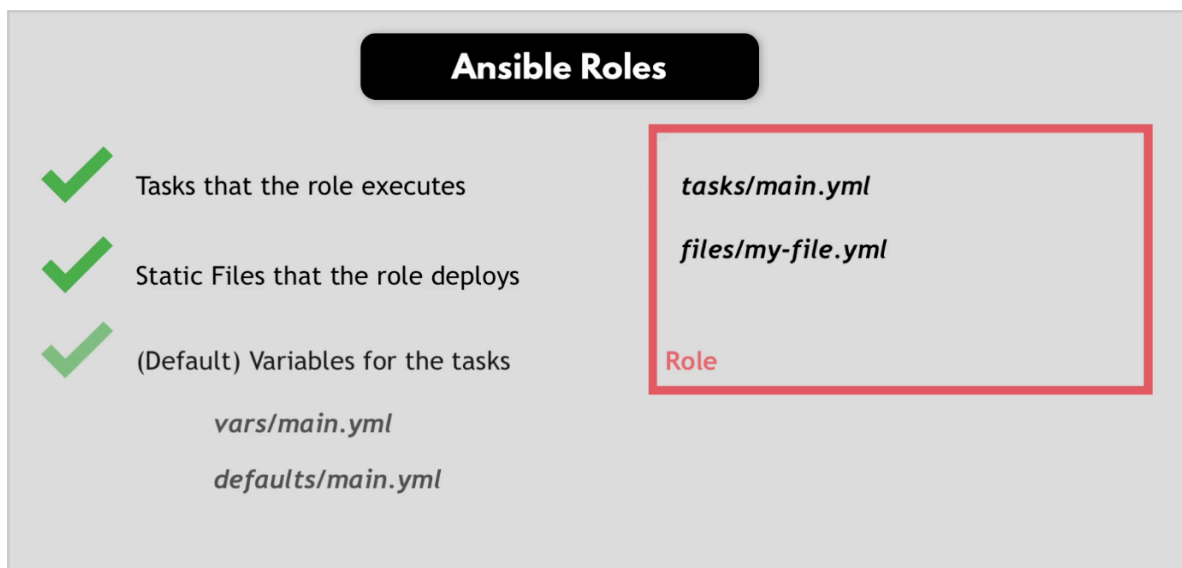
tasks/main.yml

Role

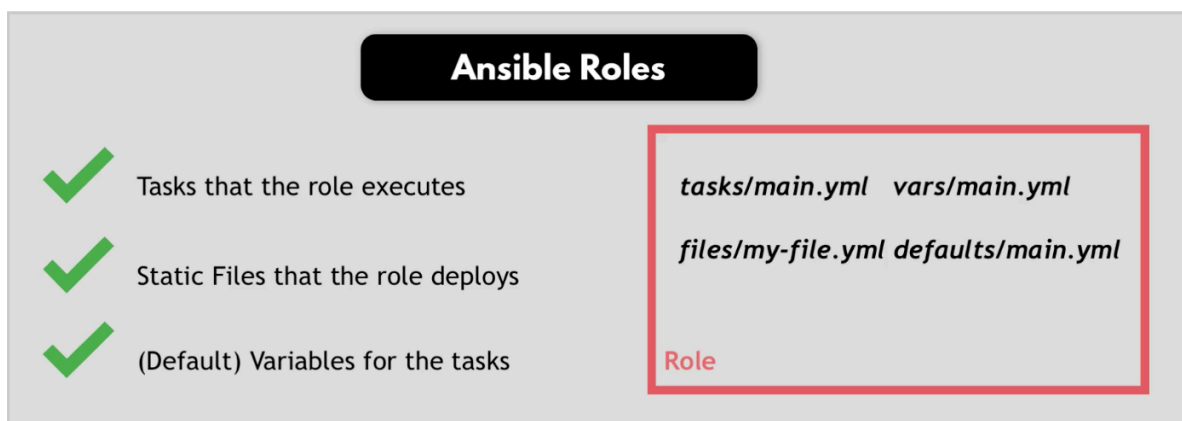
Then we can have those static files inside the role and then use them in the task



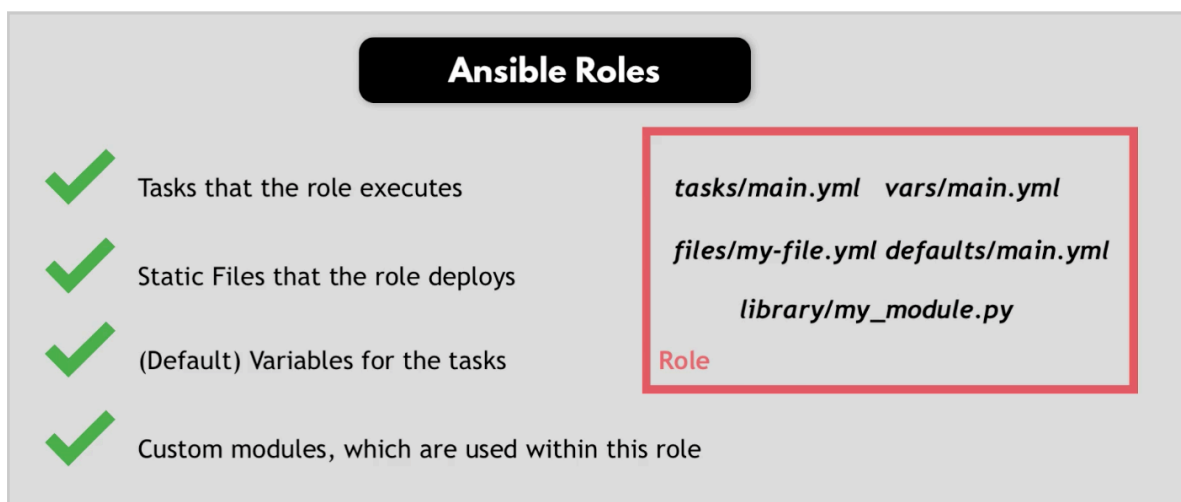
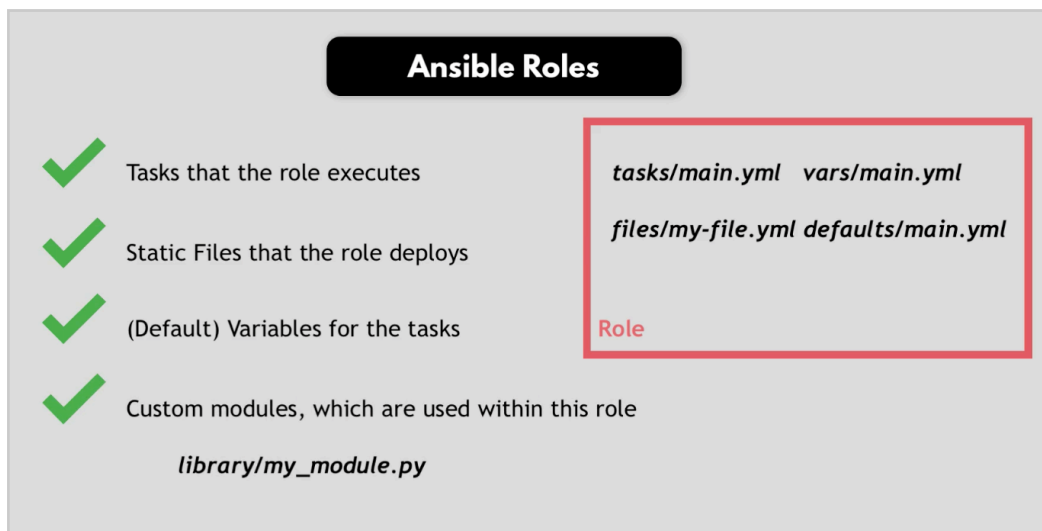
We can also define variables for the tasks



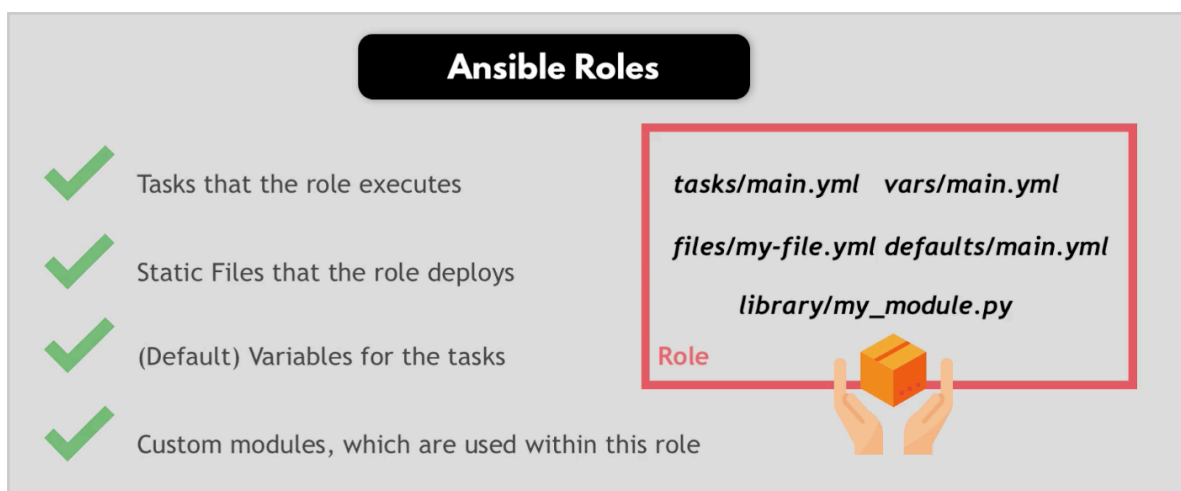
Inside the role package



Also custom modules



So now we have everything we need to execute the task in one package



Roles are like small applications:

- Easy to maintain
- Easy to reuse

And Roles have a standard file structure

```
# playbooks
site.yml
webservers.yml
fooservers.yml
roles/
    common/
        tasks/
        handlers/
        library/
        files/
        templates/
        vars/
        defaults/
        meta/
    webservers/
        tasks/
        defaults/
        meta/
```

Every role creator has to follow the standard structure above.

So if we need to define variables we know where that need to go:

Ansible Roles

- ▶ Like small applications
- ▶ Easy to maintain and reuse
- ▶ Standard file structure

easy to navigate
 easy to maintain

```
# playbooks
site.yml
webservers.yml
fooservers.yml
roles/
  common/
    tasks/
    handlers/
    library/
    files/
    templates/
    vars/
    defaults/
    meta/
  webservers/
    tasks/
    defaults/
    meta/
```

rs/main.yml

faults/main.yml

odule.py

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

This can result into turning 100 playbook into 20 playbooks.

In the roles we can define default variables

Ansible Roles

- ▶ Like small applications
- ▶ Easy to maintain and reuse
- ▶ Standard file structure

Default Variables

tasks/main.yml

vars/main.yml

files/my-file.yml

defaults/main.yml

library/my_module.py

Role

So we can parameterise the whole role!

So the role can be used without much configuration.

Also the default values can be overwritten .

Ansible Roles

- ▶ Like small applications
- ▶ Easy to maintain and reuse
- ▶ Standard file structure

tasks/main.yml *vars/main.yml*

files/my-file.yml *defaults/main.yml*

library/my_module.py

Role



Default Variables

-  Parameterize role, but execute without having to define variables
-  Possibility to overwrite default variables

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved.

Another advantage is that we can develop and test the role separately.

We can create our own roles but also we can use existing ones from the community. For example setting up mysql database on ubuntu server or installing and configuring nginx on linux servers

Ansible Roles



my_role



role_A

role_LL

role_CC

role_XY

....

role_RX

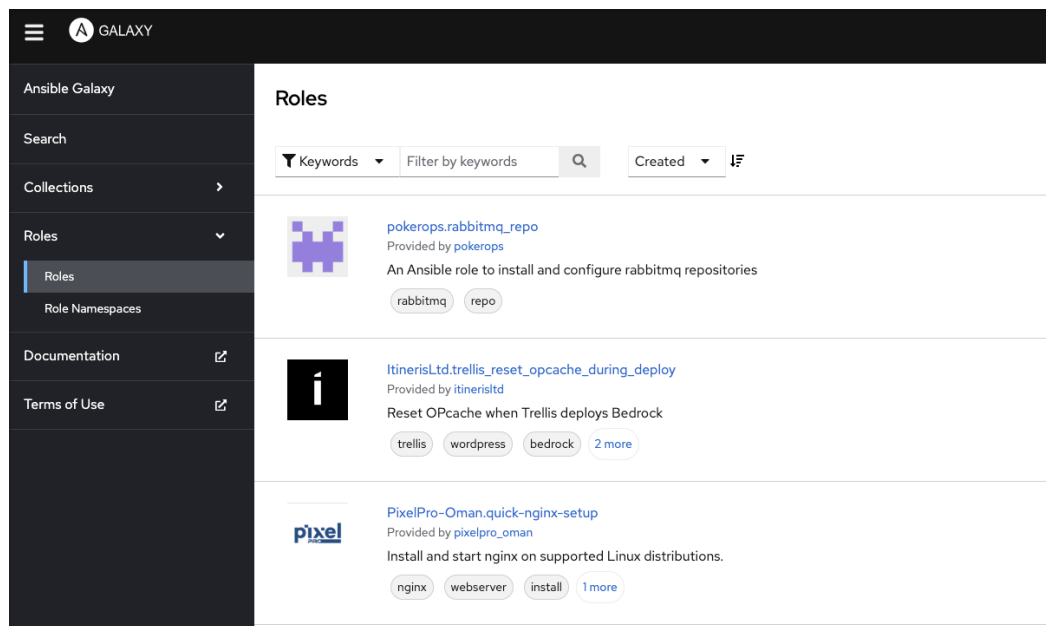
- ▶ Use existing Roles from community





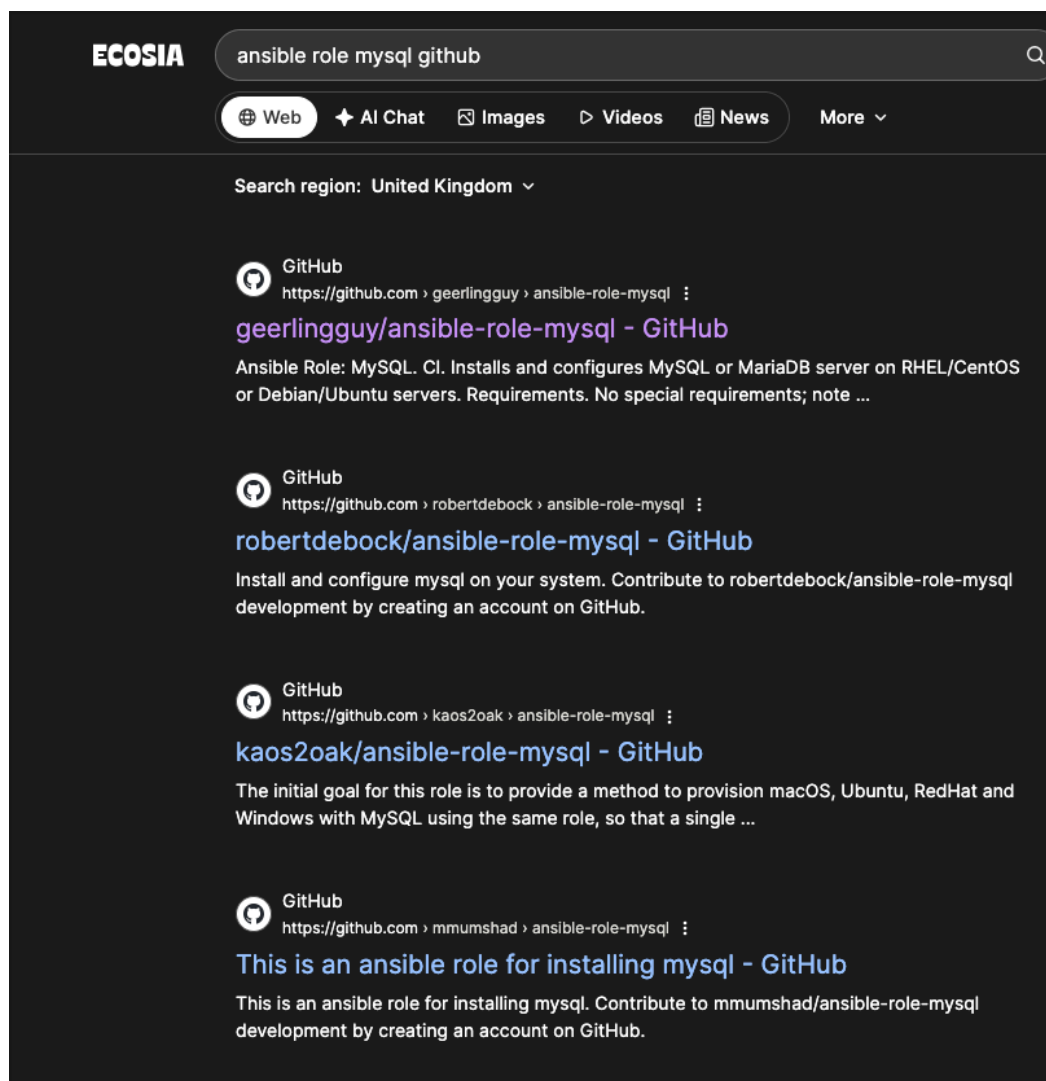
We can find the community roles in two places:

- main hub of roles -> ansible galaxy <https://galaxy.ansible.com/ui/>

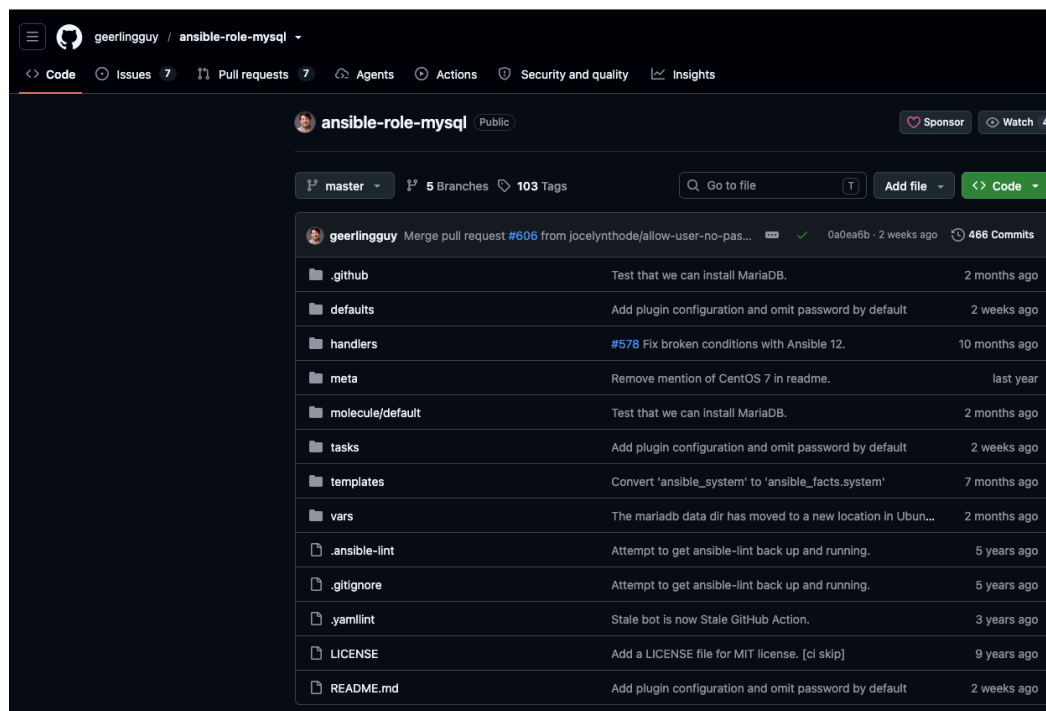


– Git repos

For example if we search for 'ansible role mysql' we get several options



And we can see that they follow the standard structure



Create Ansible Roles

Let's look at 'deploy-docker-new-user.yaml' configuration for EC2 sever.

If we want then run the playbook for a digital Ocean droplet we will have to write another playbook because some of the steps will be different. Creating the new user and starting docker containers will be the same or mostly the same.

So we can extract the tasks that are the same for EC2 and for the droplet (common to any server) so we don't duplicate the code and package them into roles.

Like Functions

- ▶ Extract common logic
- ▶ Use function in different places with different parameters

Roles are like functions. We can override values of a function via parameters but the logic stays the same.

Let's create a copy of the yaml file and rename the copy to 'deplo-docker-with-roles.yaml'

The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure for a project named 'PythonProject' under the user 'astrid'. The file 'deploy-docker-with-roles.yaml' is selected. The code editor shows the content of this file, which is an Ansible playbook. It starts with '---' and then defines a play with the name 'Wait for SSH connection', hosts 'aws_ec2', and 'gather_facts' set to 'False'. It then defines a task named 'Ensure SSH port open' with 'wait_for' set to 'port: 22', 'delay: 10', and 'timeout: 100'.

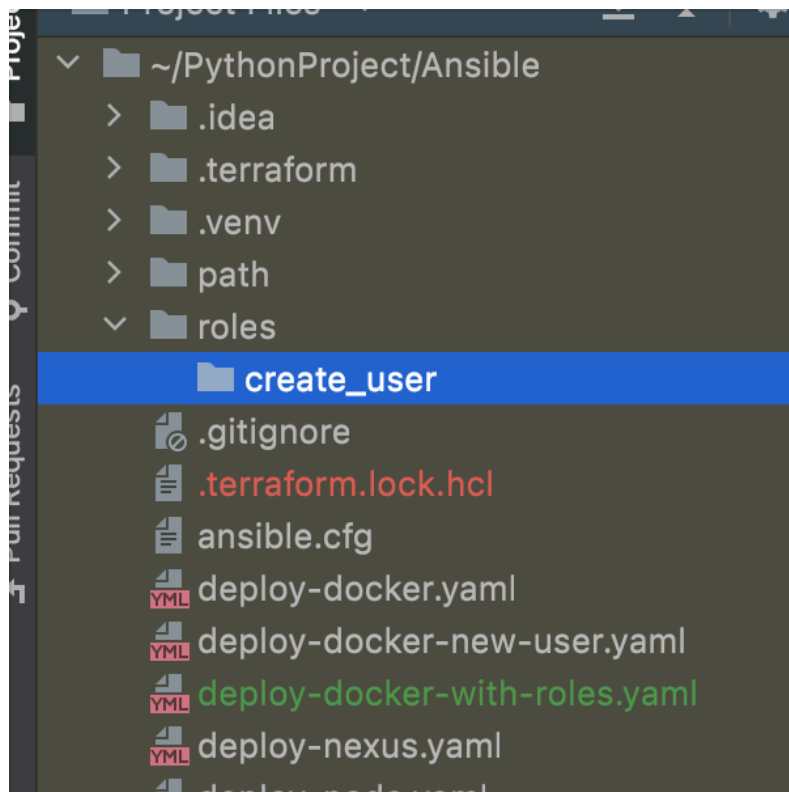
```
1 ---
2 - name: Wait for SSH connection
3   hosts: aws_ec2
4   gather_facts: False
5   tasks:
6     - name: Ensure SSH port open
7       wait_for:
8         port: 22
9         delay: 10
10        timeout: 100
```

Now we need to create a project inside the ansible project called 'roles' following the standard structure

The diagram is titled 'Ansible Roles'. It lists three key features: 'Like small applications', 'Easy to maintain and reuse', and 'Standard file structure'. The 'Standard file structure' is highlighted with two green checkmarks and the text 'easy to navigate' and 'easy to maintain'. A central box shows a file tree structure: '# playbooks', 'site.yml', 'webservers.yml', 'fooservers.yml', 'roles/' (indicated by a red box), and under 'roles/' are 'common/' (with subfolders 'tasks/', 'handlers/', 'library/', 'files/', 'templates/', 'vars/' (highlighted in blue), 'defaults/', 'meta/'), 'webservers/' (with subfolders 'tasks/', 'defaults/', 'meta/'), and 'fooservers/' (with subfolders 'tasks/', 'defaults/', 'meta/'). To the right of the tree, a red box highlights 'rs/main.yml', 'faults/main.yml', and 'odule.py'. At the bottom, it says 'Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved'.

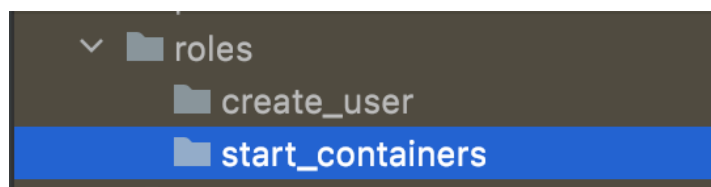
And inside the 'roles' folder we will have folders for each of the roles.

Let's extract 'create new linux user' task so I am going to create a new folder called 'create_user' inside 'roles' folder

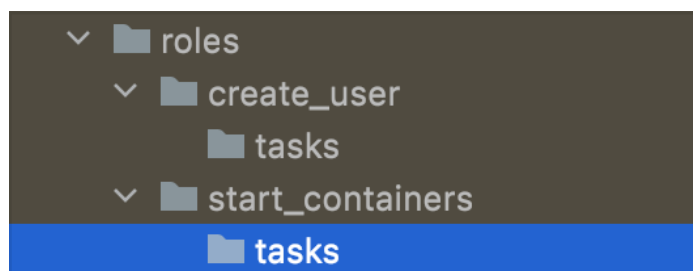


So the name of the roles is 'create_user' and we can name the role however we want.

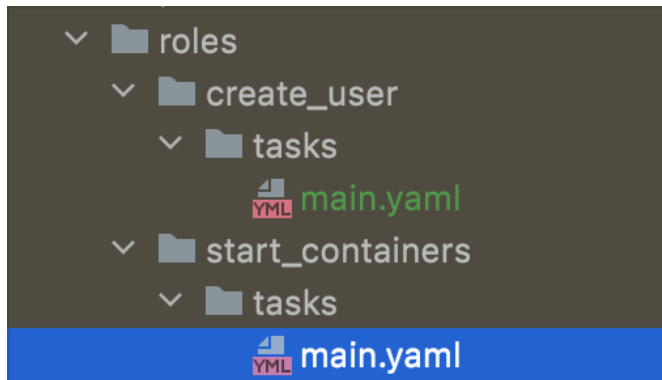
And let's create another role for the task 'Start docker containers' called 'start_containers'



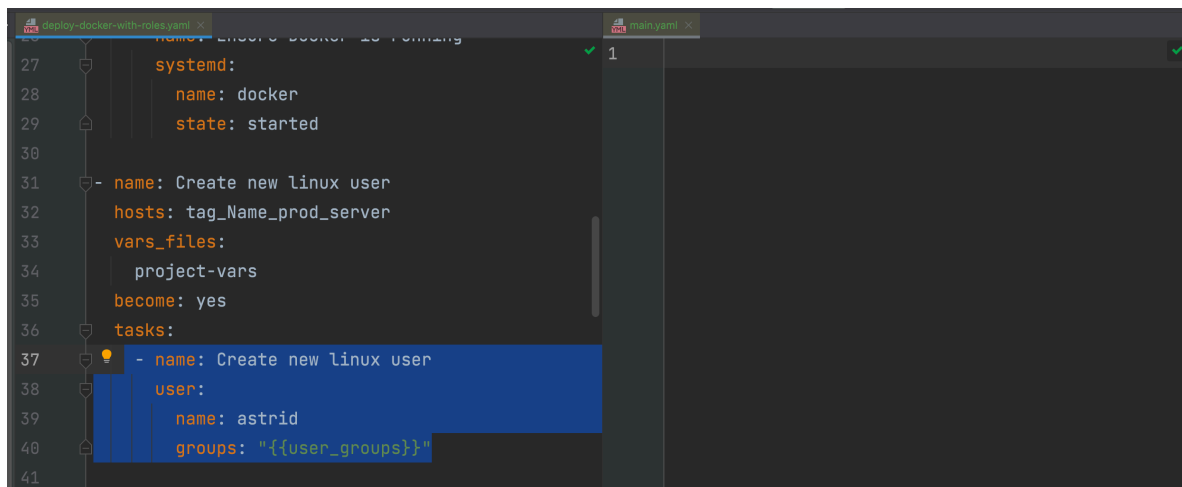
Inside each role's folder we have to have at least the folder 'tasks' so let's add this folder to each role:



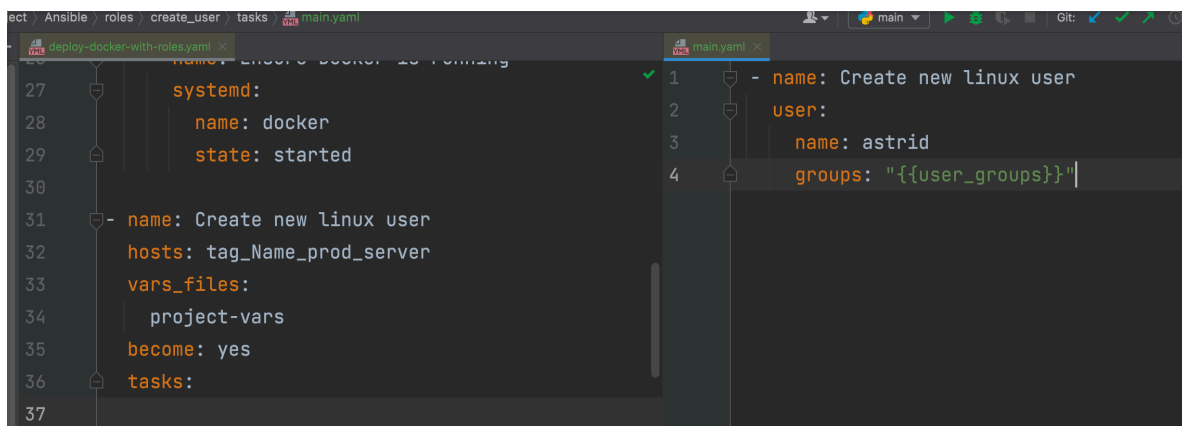
And inside each 'tasks' folder we need a file called 'main.yaml'



Now let's restructure the playbook tasks to roles in ansible.



We take the 'tasks:' code and move it from the playbook to the main.yaml (related to the create_user folder)



Now we do the same for start_containers role:

```
Users astrid PythonProject Ansible deploy-docker-with-roles.yml
48 path: ~/.docker/cli-plugins
49 state: directory
50 - name: get architecture of remote machine
51 shell: uname -m
52 register: remote_arch
53 - name: Install docker-compose
54 get_url:
55 url: https://github.com/docker/compose/releases/download
56 dest: ~/.docker/cli-plugins/docker-compose
57 mode: +x
58
59 - name: Start docker containers
60 hosts: tag_Name_prod_server
61 become: yes
62 become_user: astrid
63 vars_files:
64 project-vars
65 tasks:
66
```

```
main.yml
1 - name: Copy docker compose
2 copy:
3 src: /Users/astrid/bootcamp-java-mysql-project/do
4 dest: /home/astrid/docker-compose.yaml
5 - name: Docker login
6 docker_login:
7 username: cabfish00
8 password: "{{docker_password}}"
9 - name: Start containers from compose
10 community.docker.docker_compose_v2:
11 project_src: /home/astrid
```

Using roles in Plays

Now we need to reference the roles inside the playbook.

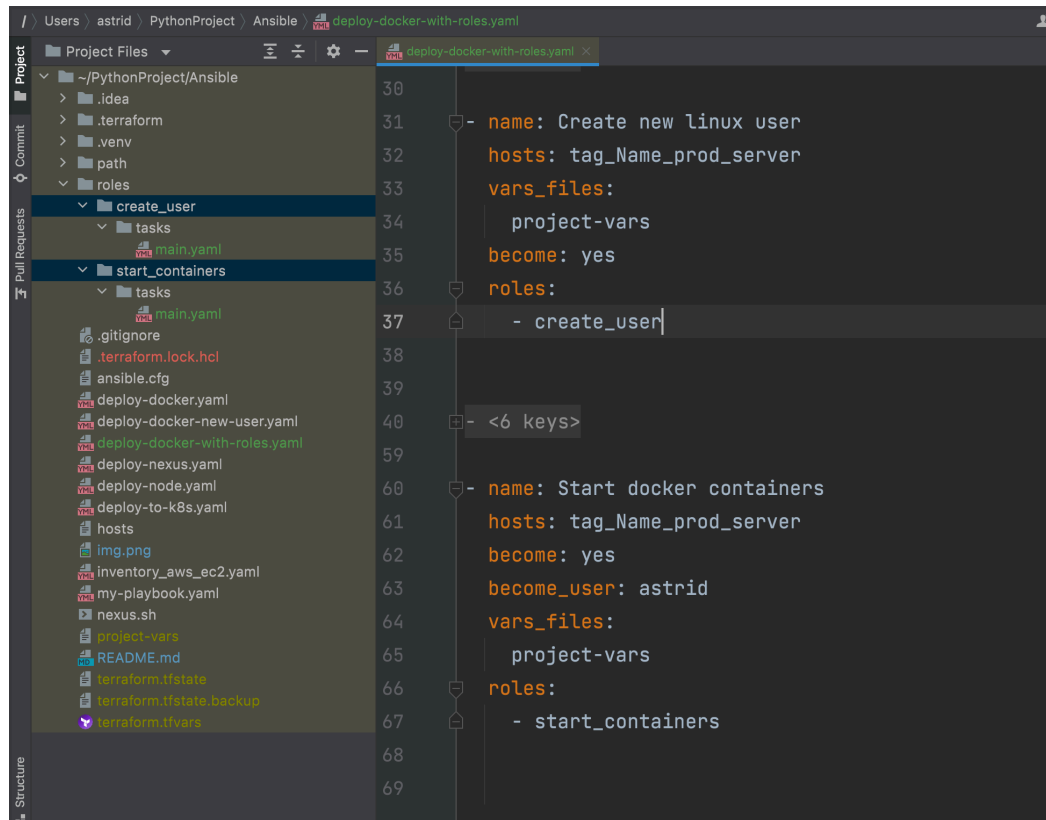
In the play we replace 'tasks:' for 'roles:' see lines 36 and 65 📌

```

31  - name: Create new linux user
32    hosts: tag_Name_prod_server
33    vars_files:
34      - project-vars
35    become: yes
36    roles:
37
38
39  - <6 keys>
58
59  - name: Start docker containers
60    hosts: tag_Name_prod_server
61    become: yes
62    become_user: astrid
63    vars_files:
64      - project-vars
65    roles:
66

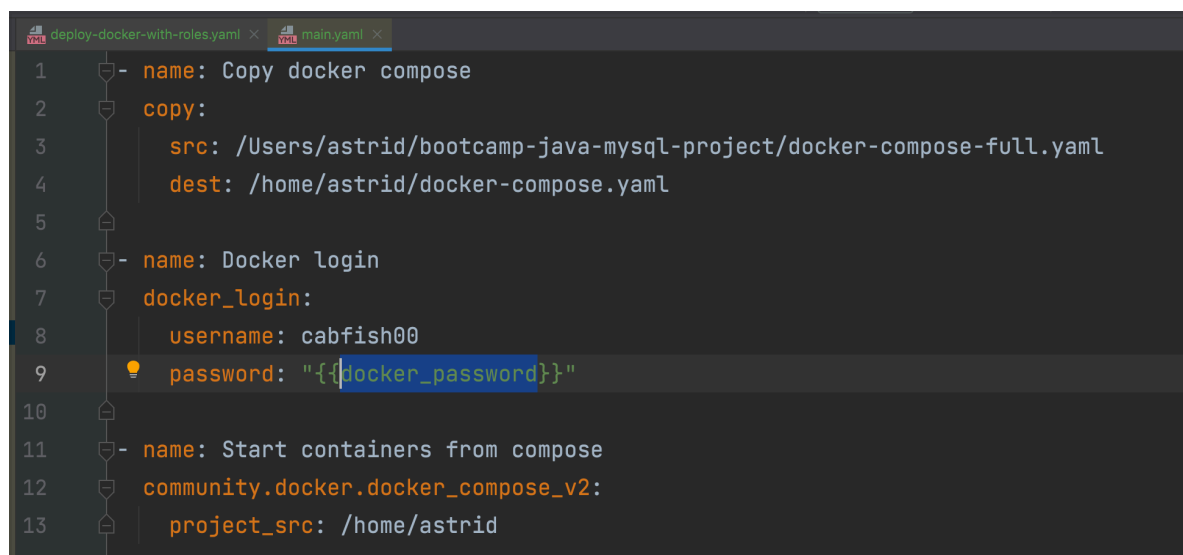
```

And we list the roles that we have in our project, so we type the name of the role:



Ansible knows how to find the yaml file because of the structure of the folders.

Now let's look at the role 'start_containers' again and we can see that in line 9 we have a variable 'docker_password' and its value is defined in the 'project-vars' file



And this will work because in the play we reference 'project-vars' file

```

60 - name: Start docker containers
61   hosts: tag_Name_prod_server
62   become: yes
63   ⚡ become_user: astrid
64   vars_files:
65     project-vars
66   roles:
67     - start_containers
68

```

Let's test the playbook using the roles and for that we will deploy infra using terraform from this branch https://github.com/AstridCaballero/Module_15_Chapter_18-19_Terraform/blob/Module_15/Ansible_chapter-19_Terraform/main.tf but only deploy two servers.

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

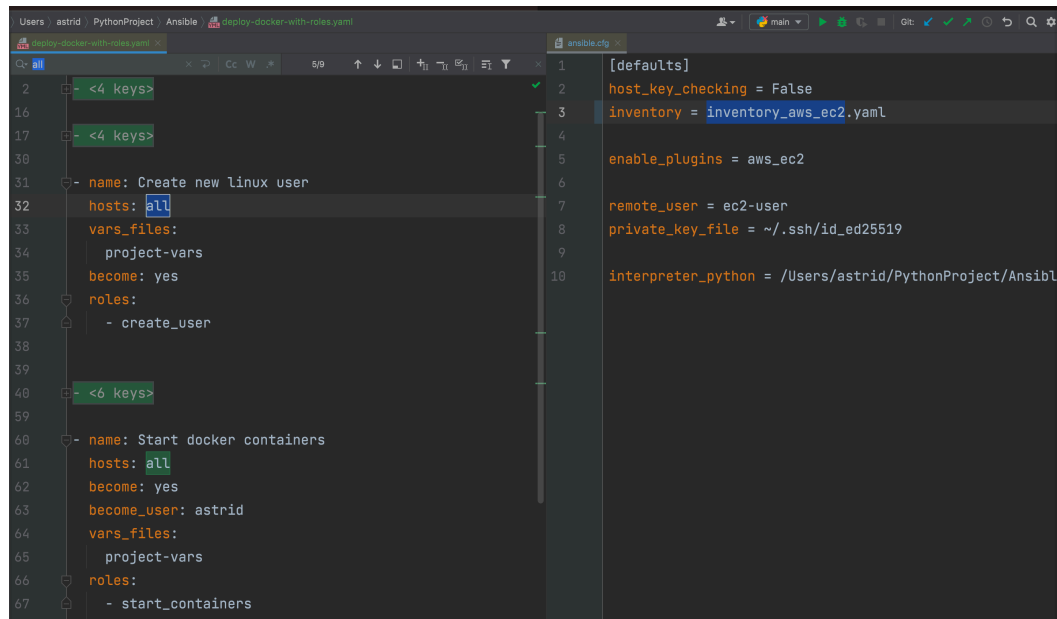
```

aws-ami_id = "ami-01c952cfc86b7870d"
ec2-public_ip_1 = "16.61.222.171"
ec2-public_ip_2 = "3.10.212.178"

```

☐	dev-server-1	i-058bd04a112ede210	Running	t3.small	3/5 checks passed	eu-west-2a	ec2-16-61-222-171.eu-...	16.61.222.171	--	--
☐	dev-server-2	i-04b7ca75cb82ac01a	Running	t3.small	3/5 checks passed	eu-west-2a	ec2-3-10-212-178.eu-w...	3.10.212.178	--	--

And make sure that ansible uses the dynamic inventory

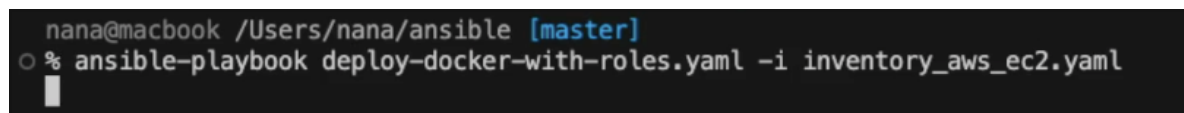


The image shows a code editor with two files open. The left file is 'deploy-docker-with-roles.yaml' and the right file is 'ansible.cfg'.

```
2  - <4 keys>
16 - <4 keys>
17 - <4 keys>
31 - name: Create new linux user
32   hosts: all
33   vars_files:
34     - project-vars
35   become: yes
36   roles:
37     - create_user
38
39 - <6 keys>
59 - name: Start docker containers
60   hosts: all
61   become: yes
62   become_user: astrid
63   vars_files:
64     - project-vars
65   roles:
66     - start_containers
67
```

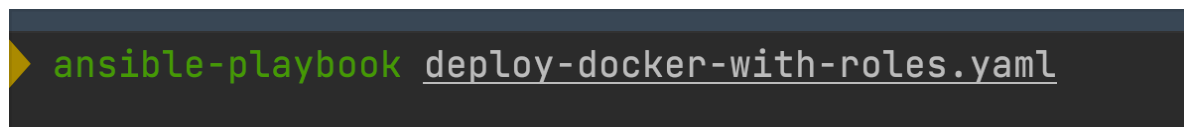
```
1 [defaults]
2 host_key_checking = False
3 inventory = inventory_aws_ec2.yaml
4
5 enable_plugins = aws_ec2
6
7 remote_user = ec2-user
8 private_key_file = ~/.ssh/id_ed25519
9
10 interpreter_python = /Users/astrid/PythonProject/Ansibl
```

Nana pass the inventory file as a parameter



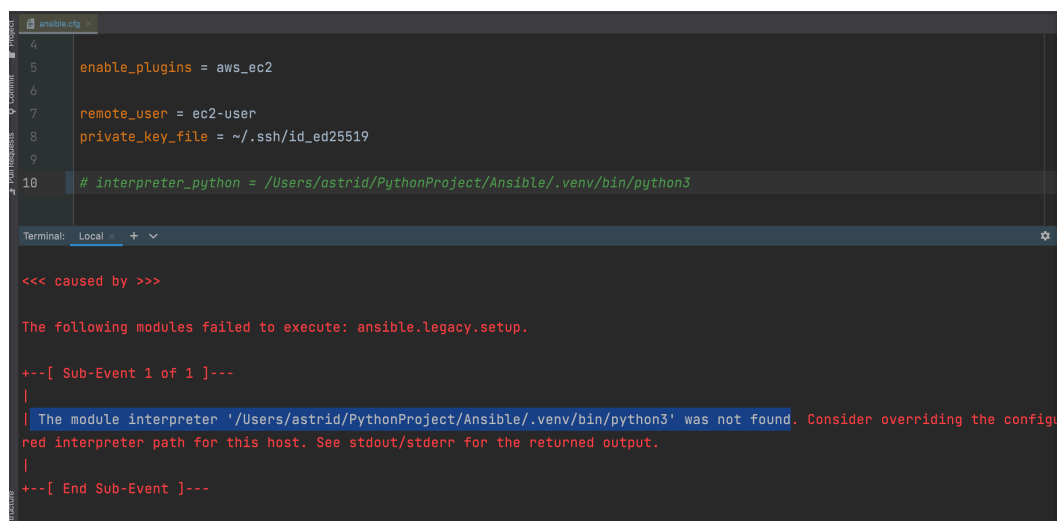
```
nana@macbook /Users/nana/ansible [master]
% ansible-playbook deploy-docker-with-roles.yaml -i inventory_aws_ec2.yaml
```

I didn't because I updated the ansible.cfg



```
ansible-playbook deploy-docker-with-roles.yaml
```

I got an error because I was python was not found



The image shows the 'ansible.cfg' file in the editor and a terminal window below it.

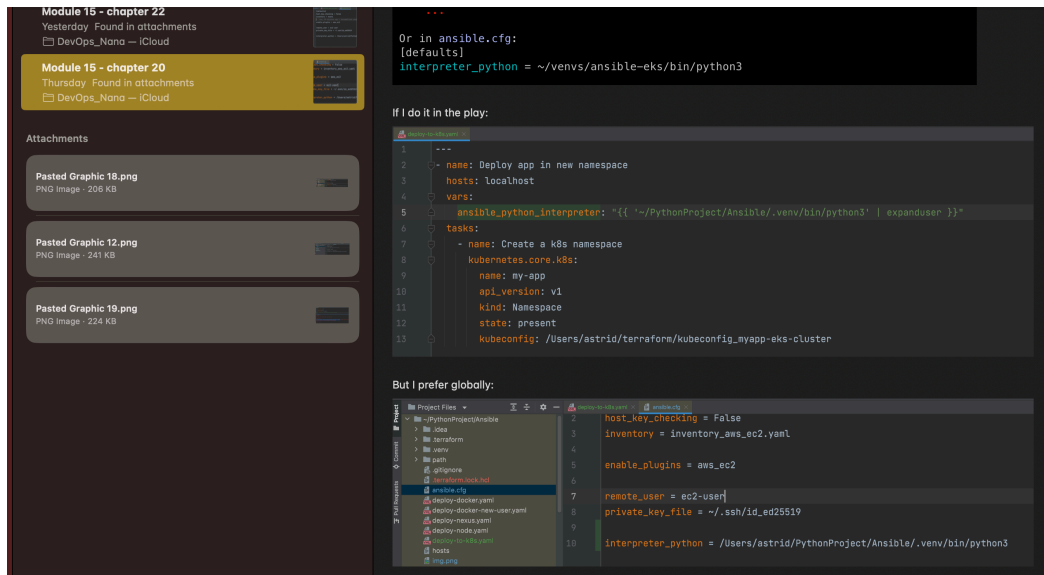
```
4
5 enable_plugins = aws_ec2
6
7 remote_user = ec2-user
8 private_key_file = ~/.ssh/id_ed25519
9
10 # interpreter_python = /Users/astrid/PythonProject/Ansible/.venv/bin/python3
```

```
<<< caused by >>>

The following modules failed to execute: ansible.legacy.setup.

---[ Sub-Event 1 of 1 ]---
| The module interpreter '/Users/astrid/PythonProject/Ansible/.venv/bin/python3' was not found. Consider overriding the configu
red interpreter path for this host. See stdout/stderr for the returned output.
|
---[ End Sub-Event ]---
```

I had set the python interpreter of my Mac which I need it for a previous demo, Module 15 - chapter 20 when the host was 'localhost' and we run ansible from our laptop instead of a remote server.



But now ansible is running the command on the remote server EC2 instance and EC2 instance has python3 in a different location.

Fix options:

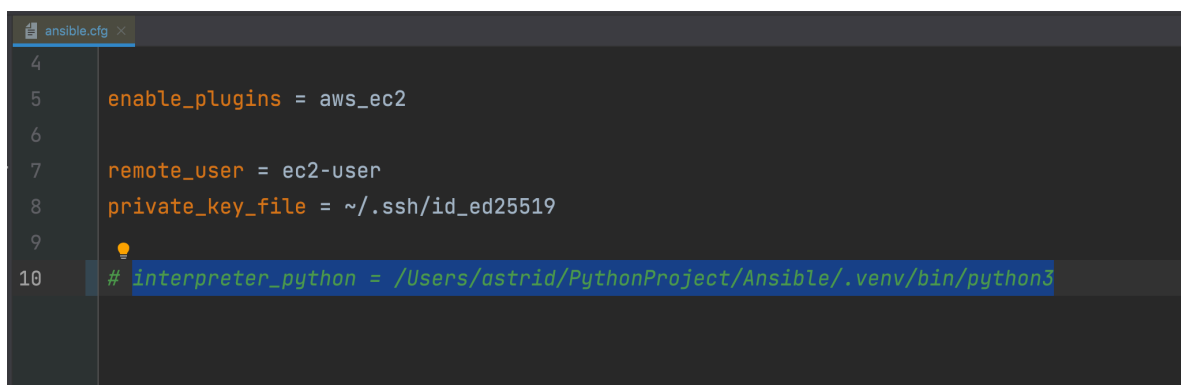
1. Remove it entirely and let Ansible auto-discover Python on the remote host (recommended for standard AMLs):

just delete the ansible_python_interpreter line

2. Point it at the actual remote interpreter, e.g. for Amazon Linux/Ubuntu typically:

ansible_python_interpreter: /usr/bin/python3

For now I will let ansible to auto-discover python in the EC2 server so I have commented it out.



Let's run the playbook again

-> ansible-playbook deploy-docker-with-roles.yml

```
TASK [start_containers : Start containers from compose] *****
changed: [ec2-3-10-212-178.eu-west-2.compute.amazonaws.com]
changed: [ec2-16-61-222-171.eu-west-2.compute.amazonaws.com]

PLAY RECAP *****
ec2-16-61-222-171.eu-west-2.compute.amazonaws.com : ok=14  changed=9    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
ec2-3-10-212-178.eu-west-2.compute.amazonaws.com : ok=14  changed=9    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

Check one of the EC2 instances by sshing into it

```
PLAY RECAP *****
ec2-16-61-222-171.eu-west-2.compute.amazonaws.com : ok=14    changed=9
ignored=0
```

```
-> ssh ec2-user@ec2-16-61-222-171.eu-west-2.compute.amazonaws.com
-> sudo docker ps
```

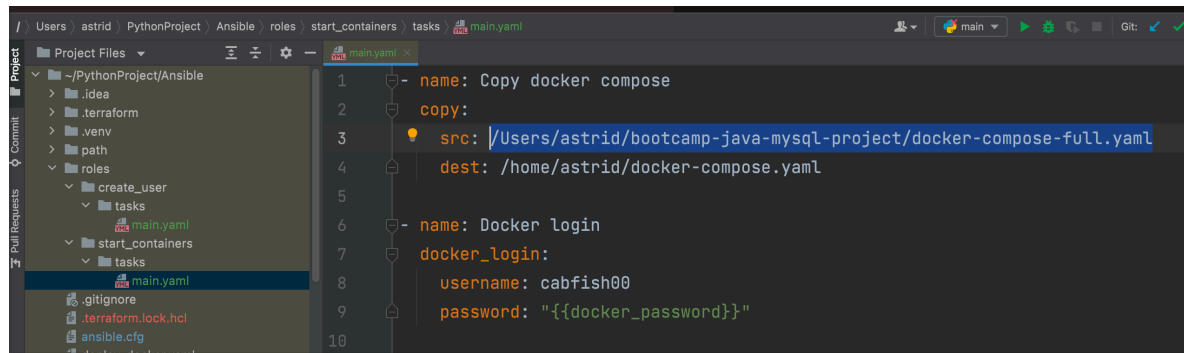
[illegible]

And the 3 containers are running 

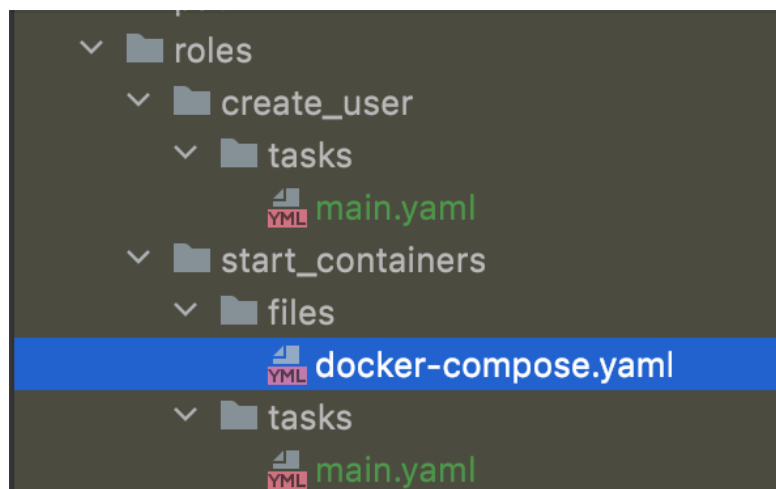
Complete our Roles

Remember we can package more things inside the role and the minimum requirement is the 'tasks' folder.

For example let's say we don't want to reference the docker-compose file from `/Users/astroid/bootcamp-java-mysql-project/docker-compose-full.yaml` instead we want to have the file packaged in the 'roles' folder



So let's create a new folder in the role 'start_containers' called 'files' as per the ansible roles standard, it must be called 'files'

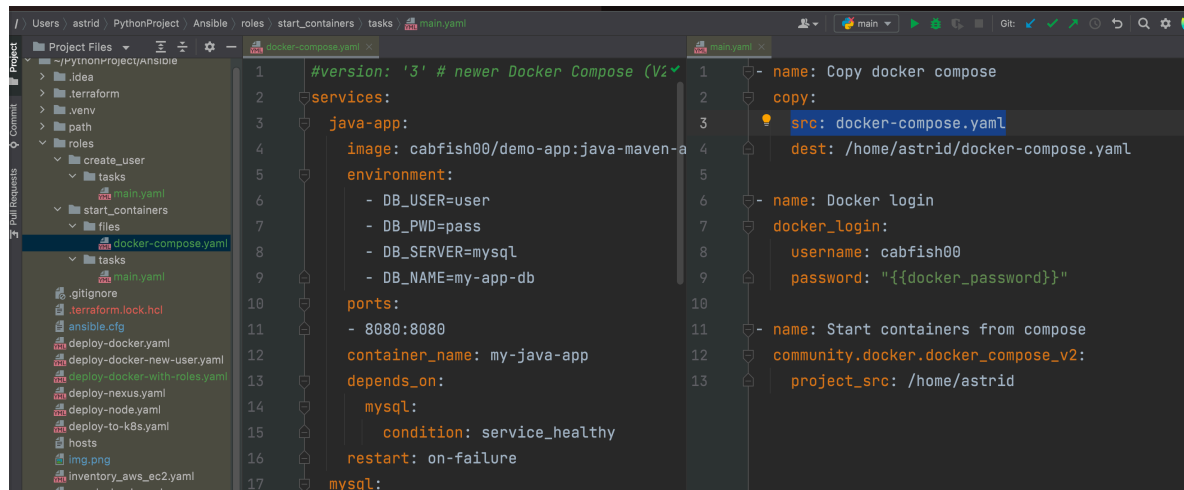


And inside the 'files' folder I create the docker-compose.yaml file and copy and paste the content in there from the original docker-compose.yaml file.

```

cat /Users/astrid/bootcamp-java-mysql-project/docker-compose-full.yaml
#version: '3' # newer Docker Compose (V2) no longer needs the top-level version: '3.x' field in the compose file and just ignores it
services:
  java-app:
    image: cabfish00/demo-app:java-maven-ansible-2.0
    environment:
      - DB_USER=user
      - DB_PWD=pass
      - DB_SERVER=mysql
      - DB_NAME=my-app-db
    ports:
      - 8080:8080
    container_name: my-java-app
    depends_on:
      mysql:
        condition: service_healthy
    restart: on-failure
  mysql:
    image: mysql
    ports:
      - 3306:3306
    environment:
      - MYSQL_ROOT_PASSWORD=my-secret-pw
      - MYSQL_DATABASE=my-app-db
      - MYSQL_USER=user
      - MYSQL_PASSWORD=pass
    volumes:
      - mysql-data:/var/lib/mysql
    container_name: mysql
    # command: --default-authentication-plugin=mysql_native_password
    healthcheck:
      test: [ "CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-pmy-secret-pw" ]
      interval: 5s
      timeout: 5s
      retries: 10
  phpmyadmin:

```



And we update the reference to the file in main.yml to docker-compose.yml in line 3 🙌

And we don't have to tell ansible the path like file/docker-compose.yml -> ansible knows the file is in the 'files' folder.

Let's execute the playbook again:

-> ansible-playbook deploy-docker-with-roles.yml

```

TASK [start_containers : Start containers from compose] *****
ok: [ec2-16-61-222-171.eu-west-2.compute.amazonaws.com]
ok: [ec2-3-10-212-178.eu-west-2.compute.amazonaws.com]

PLAY RECAP *****
ec2-16-61-222-171.eu-west-2.compute.amazonaws.com : ok=14  changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ec2-3-10-212-178.eu-west-2.compute.amazonaws.com : ok=14  changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

And it works!

Customise Roles with Variables

We can define variables inside the roles.

For example, let's say we hardcode the registry url, the registry where we store our docker images like docker hub.

https://docs.ansible.com/projects/ansible/latest/collections/community/docker/docker_login_module.html

[Collection Index](#) / [Collections in the Community Namespace](#) / [Community.Docker](#) / community.docker **docker_login** module – Log into a Docker registry

This is the **latest** (stable) Ansible community documentation. For Red Hat Ansible Automation Platform subscriptions, see [Life Cycle](#) for version details.

community.docker.docker_login module – Log into a Docker registry

Note

This module is part of the `community.docker` collection (version 5.2.2).

You might already have this collection installed if you are using the `ansible` package. It is not included in `ansible-core`. To check whether it is installed, run `ansible-galaxy collection list`.

To install it, use: `ansible-galaxy collection install community.docker`. You need further requirements to be able to use this module, see [Requirements](#) for details.

To use it in a playbook, specify: `community.docker.docker_login`.

registry_url aliases: registry, url string	The registry URL. Default: <code>"https://index.docker.io/v1/"</code>
---	---

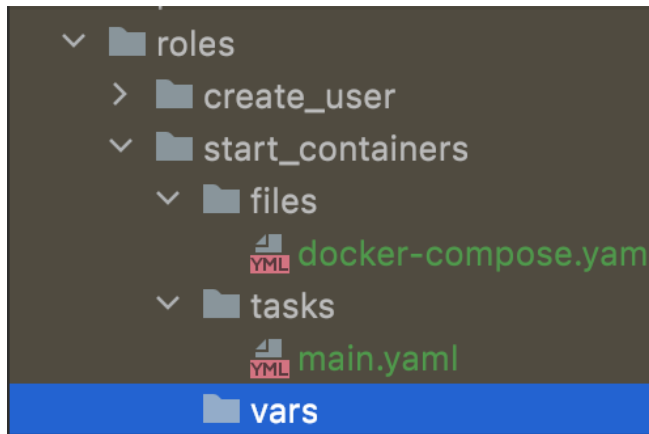
```

← →  ansible
! deploy-docker-with-roles.yaml U  ! main.yaml .../create_user/... U  ! main.yaml .../start_containers/... U X
1  - name: Copy docker compose
2    copy:
3      src: docker-compose.yaml
4      dest: /home/nana/docker-compose.yaml
5
6  - name: Docker login
7    docker_login:
8      registry_url: https://index.docker.io/v1/
9      username: nanatwn
10     password: "{{docker_password}}"
11
12  - name: Start containers from compose
13    community.docker.docker_compose_v2:
14      project_src: /home/nana
  
```

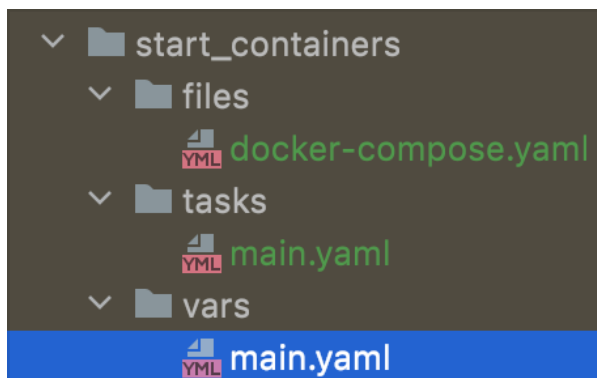
If we hardcode the registry then if we want to use the role to fetch a docker image from a nexus registry it will fail **!!**

With variables we can give flexibility to the roles, to be able to use different values 💡

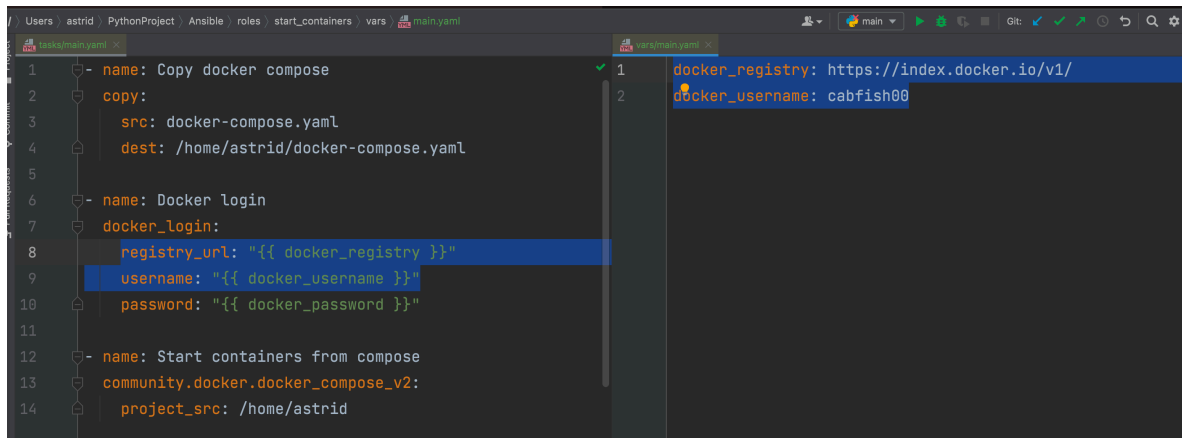
To hold the variables we create a new folder inside our role called 'vars'



And inside 'vars' folder we create a file called 'main.yaml'



And create the variables in the 'tasks' folder and set the values in the 'vars' folder



There are several ways to set the variables values, like using the 'project-vars' file or using the role's 'vars' folder.

We can have the same variable value set in both the 'project-vars' file or using the role's 'vars' folder and ansible has precedence rules to figure out which value to use.

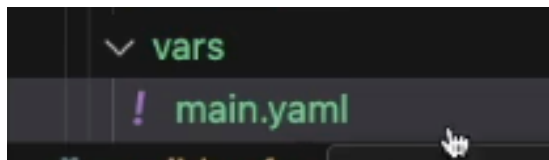
Variable Precedence

from least to greatest (the last listed variables override all other variables)

1. command line values (for example, `-u my_user`, these are not variables)
2. role defaults (defined in `role/defaults/main.yml`)
3. inventory file or script group vars
4. inventory group_vars/all
5. playbook group_vars/all
6. inventory group_vars/*
7. playbook group_vars/*
8. inventory file or script host vars
9. inventory host_vars/*
10. playbook host_vars/*
11. host facts / cached set_facts
12. play vars
13. play vars_prompt
14. play vars_files
15. role vars (defined in `role/vars/main.yml`)
16. block vars (only for tasks in block)
17. task vars (only for the task)
18. include_vars
19. set_facts / registered vars
20. role (and include_role) params
21. include params
22. extra vars (for example, `-e "user=my_user"`) (always win precedence)

So vars passed with flag `-e` in the command line will win precedence over everything (22 🙌)

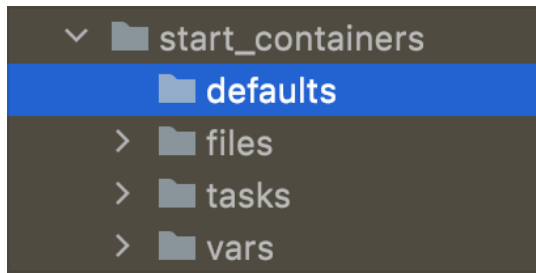
And the next most important is the value set in 'vars' file of the role



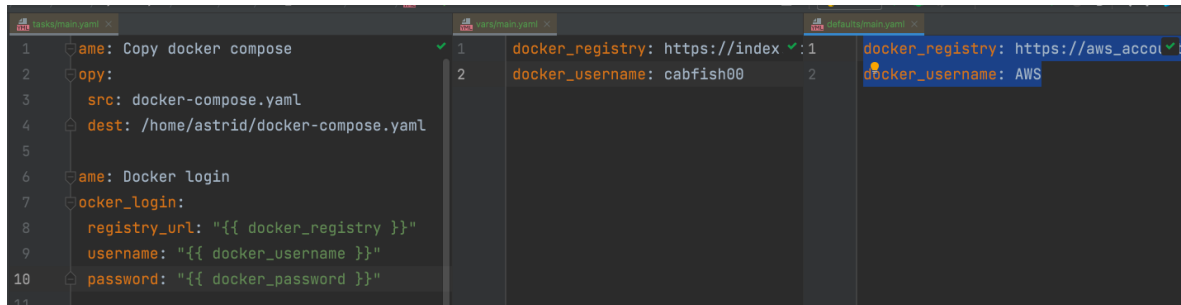
For example if 'vars/main.yml' has `docker_password`. Then the value is set in two places 'vars/main.yml' and 'project-vars' and 'vars/main.yml' will take precedence.

Now the ansible user can customise the role 'start_containers' and the creator of the role can also define default values to the role.

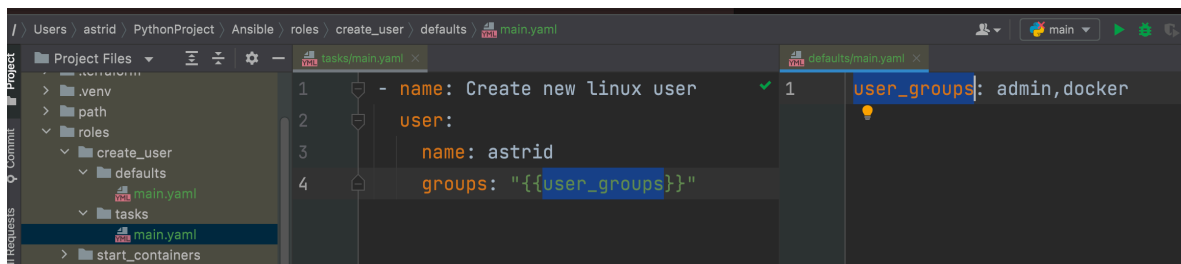
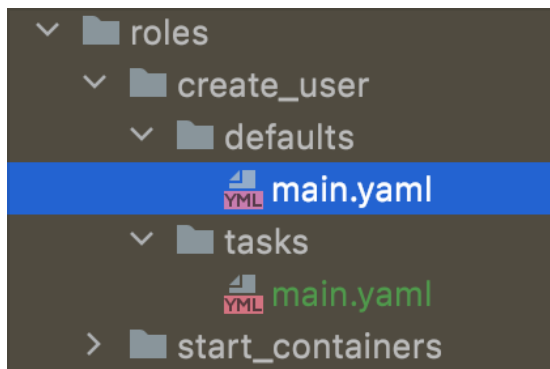
So if the user doesn't set the values in the 'vars/main.yml' or the 'project-vars' file or pass them as command line arguments, so the user doesn't customise the variables then the default values for the variables will be used and to set the default values we create a folder called 'defaults'



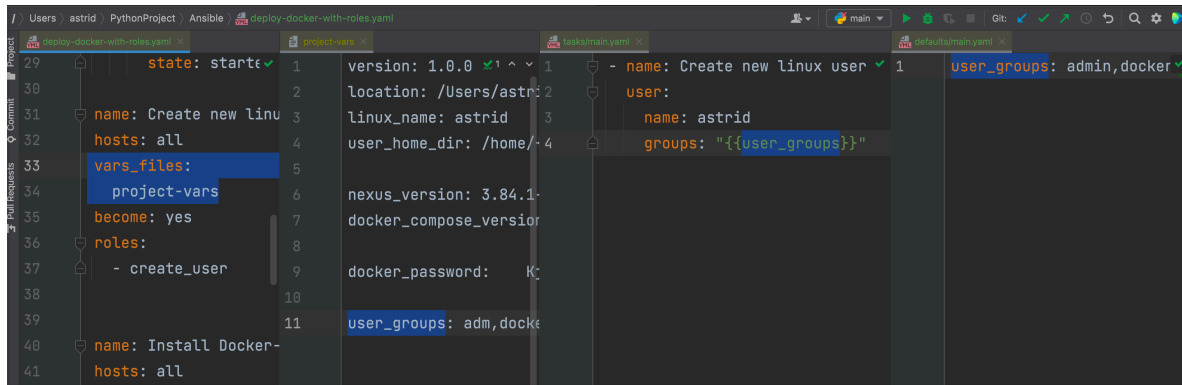
And in the 'defaults' folder we create a file called 'main.yaml' file and set the default values there



Let's create default variables for 'create_user' role as well



So this is what we have so far regarding the user groups to create a new linux user



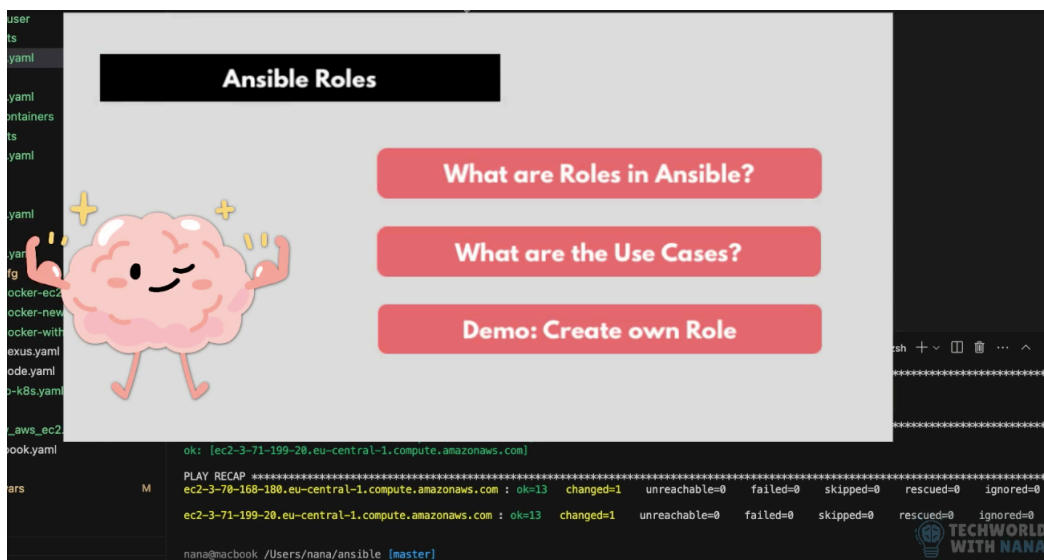
Currently the ansible playbook is telling the task to use the 'project-vars' values, so the 'defaults' won't be used as they have low preference.

Let's test the playbook

-> ansible-playbook deploy-docker-with-roles.yaml

```
TASK [start_containers : Start containers from compose] *****
ok: [ec2-16-61-222-171.eu-west-2.compute.amazonaws.com]
ok: [ec2-3-10-212-178.eu-west-2.compute.amazonaws.com]

PLAY RECAP *****
ec2-16-61-222-171.eu-west-2.compute.amazonaws.com : ok=14  changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ec2-3-10-212-178.eu-west-2.compute.amazonaws.com : ok=14  changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```



The ROLES becomes very valuable the bigger and more complex the ansible projects are.

